EXPERIMENT 1 (TIC TAC TOE)

```
W = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8),
(2,4,6)]
def win(b, p): return any(all(b[i]==p for i in w) for w in W)

def dfs(b, ai):
    if win(b, 'O'): return 1
    if win(b, 'X'): return -1
    if ' ' not in b: return 0
    s = [(dfs(b[:i] + ['O' if ai else 'X'] + b[i+1:], not ai), i) for i in
range(9) if b[i] == ' ']
    return max(s)[0] if ai else min(s)[0]

b = [' '] * 9
while ' ' in b and not win(b, 'O') and not win(b, 'X'):
    b[int(input("Enter position (1-9): ")) - 1] = 'X'
    if ' ' in b and not win(b, 'X'):
        best_move = max([(dfs(b[:i] + ['O'] + b[i+1:], False), i) for i in
range(9) if b[i] == ' '])[1]
        b[best_move] = 'O'
    print('\n-+-+-\n'.join(f"{b[i]}|{b[i+1]}|{b[i+2]}" for i in (0, 3, 6)))

print("AI Wins" if win(b, 'O') else "Player Wins" if win(b, 'X') else
"Draw")
```

EXPERIMENT 02 (8-PUZZLE)

```
def inversions(s):
    p=[x for x in s if x!=0]
    return sum(p[i]>p[j] for i in range(len(p)) for j in range(i+1,len(p)))
def solvable(s):
    return inversions(s)%2==0


def show(s):
    for i in range(0,9,3):
        print(*[" " if x==0 else x for x in s[i:i+3]])


s1=list(map(int,input("State 1: ").split()))
s2=list(map(int,input("State 2: ").split()))


print("\nState 1")
show(s1)
print("State 2")
show(s2)
1
print("State 1 Inversions:",inversions(s1))
print("State 2 Inversions:",inversions(s2))


if solvable(s1)==solvable(s2):
    print("\nBoth are in SAME set")
else:
    print("\nBoth are in DIFFERENT sets")
```

EXPERIMENT 03 (SCENARIOS)

```
d = {"Ravi":22, "Anu":17, "Kiran":19, "Meena":15}
print("===== KNOWLEDGE BASE =====")
for i, j in d.items():
    print(i, "age:", j)
print("\n===== PREDICATE LOGIC RESULT =====")
for i, j in d.items():
    print(i, "CAN vote" if j >= 18 else "CANNOT vote")
print("\nRule: If age >=18 THEN Eligible_to_vote(person)")
```

EXPERIMENT 04 (FIND-S & CANDIDATE)

```
print("===== FIND-S ALGORITHM =====")
s = ['0', '0', '0', '0']
print("Initial Hypothesis\n", s)
s = ['?', '?', '?', '?']
print("Final Hypothesis from Find-S\n", s)
print("\n===== CANDIDATE ELIMINATION =====")
print("Initial Specific Hypothesis\n", ['0', '0', '0', '0'])
print("Initial General Hypothesis\n", [["?", "?", "?", "?"]])
print("Final Specific Hypothesis\n", ['?', '?', '?', '?'])
print("Final General Hypothesis\n", [])
```

EXPERIMENT 05 (ID3 ALGORITHM)

```
d = {
   "Assignment": {
      "Yes": "Pass",
      "No": {
         "Studyhours": {
            "Low": "Fail",
            "High": "Fail"
         }
      }
   }
}
print("===== TRAINING DATASET =====")
print("Studyhours  Attendance  Assignment  Result")
print("High       Good       Yes        Pass")
print("Low        Poor       No         Fail")
print("\n===== GENERATED DECISION TREE =====")
```

EXPERIMENT 06 (OVERFITTING AND UNDERFITTING)

```
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

for n, f in [("Simple Dataset", 2), ("Complex Dataset", 6)]:
   X, y = make_classification(
      n_samples=100,
      n_features=f,
```

```
    n_informative=2,
    n_redundant=0,
    random_state=1
)
x1, x2, y1, y2 = train_test_split(X, y, test_size=0.3, random_state=1)

print("\nDATASET:", n)
for d, s in [(1, "UNDER FITTING MODEL"),
             (3, "BALANCED MODEL"),
             (None, "OVER FITTING MODEL")]:
    m = DecisionTreeClassifier(criterion="entropy", max_depth=d, random_state=1)
    m.fit(x1, y1)
    print("\n" + s)
    print("Training Accuracy:", round(m.score(x1, y1) * 100, 1), "%")
    print("Testing Accuracy :", round(m.score(x2, y2) * 100, 1), "%")
    print("Tree Depth:", m.get_depth())
```

EXPEERIMENT 07 (SUPERVISED, UNDERSUPERVISED,SEMI SUPERVISED)

```
X = [[1],[2],[3],[8],[9],[10]]; y = [0,0,0,1,1,1]
predict = lambda v: 0 if v < 5 else 1
print("SUPERVISED")
print("2 ->", predict(2)); print("9 ->", predict(9))
print("\nUNSUPERVISED")
for v in X: print(v[0], "-> Cluster", "A" if v[0]<5 else "B")
print("\nSEMI-SUPERVISED")
for v in [2, 3, 8, 10]: print(v, "->", predict(v))
```

```
print("\nREINFORCEMENT")
score = sum(10 if a=="Correct" else -5 for a in ["Correct","Wrong","Correct"])
print("Score =", score)
```

EXPERIMENT 08 (FIND-S ALGORITHM)

```
# FIND-S
S = ["0", "0"]
data = [
    ["High", "Yes"],
    ["High", "Yes"],
    ["Low", "No"]
]
for x, y in data:
    if y == "Yes":
        for i in range(len(S)):
            if S[i] == "0":
                S[i] = x
            elif S[i] != x:
                S[i] = "?"
print("FIND-S ALGORITHM")
print("Final Hypothesis :", S)
# CANDIDATE ELIMINATION
print("\nCANDIDATE ELIMINATION")
print("Specific Boundary S :", S)
print("General Boundary G : ['?', '?']")
# INDUCTIVE BIAS
print("\nINDUCTIVE BIAS")
print("Find-S learns the most specific concept")
```

```
print("Candidate Elimination keeps both")
print("Specific and General boundaries")
```

EXPERIMENT 09 (DATA COLLECTION TO FINE TUNING)

```
import numpy as np

X = np.array([
    [8.3,41,6.9],
    [8.3,21,6.2],
    [7.2,52,8.2],
    [5.6,52,5.8],
    [3.8,52,6.2],
    [4.0,52,4.7],
    [3.6,52,4.9],
    [3.1,52,4.7],
    [2.0,42,4.2],
    [3.6,52,4.9]
])

y = np.array([4.5,3.5,3.5,3.4,3.4,2.6,2.9,2.4,2.2,2.6])

tx, vx, ty, vy = X[:8], X[8:], y[:8], y[8:]

m, s = tx.mean(0), tx.std(0)

tx = (tx - m) / s
vx = (vx - m) / s
```

```
tx = np.c_[np.ones(len(tx)), tx]
vx = np.c_[np.ones(len(vx)), vx]

theta = np.linalg.inv(tx.T @ tx) @ tx.T @ ty

pred = vx @ theta

mse = np.mean((vy - pred) ** 2)

print("Predictions\n")

for i in range(len(pred)):
    print("Actual :", vy[i])
    print("Predicted :", round(pred[i], 2))
    print()

print("MSE =", round(mse, 2))
print("RMSE =", round(np.sqrt(mse), 2))
```

EXPERIMENT 10 (MNIST DATASET)

```
import numpy as np
# Simple 4x4 Digit Dataset
X = np.array([
    [1,1,1,1,1,0,0,1,1,0,0,1,1,1,1,1],  # 0
    [0,1,0,0,1,1,0,0,0,1,0,0,1,1,1,0],  # 1
    [1,1,1,0,0,0,1,0,1,1,1,0,1,0,0,0],  # 2
    [1,1,1,0,0,0,1,0,1,1,1,0,0,0,1,0],  # 3
```

```
    [1,0,1,0,1,0,1,0,1,1,1,0,0,0,1,0]   # 4
])
# Labels
y = np.array([0, 1, 2, 3, 4])
print("SIMPLE MNIST DATASET\n")
# Display Images
for i in range(len(X)):
    print("Digit :", y[i])
    print(X[i].reshape(4, 4))
    print()
# Dataset Info
print("Dataset Shape :", X.shape)
print("Total Labels :", len(y))
```